

UVA MEMBRO DA REDE ILUMINO		FOLHA DE QUESTÕES					
CURSO: Ciência da Computação		DISCIPLINA: Aplicações na Internet					
ASS.: Diego Barbosa Moreira		NOME: Diego Barbosa Moreira					
DATA:	Nº de ordem	PROVA: A2	<table border="1"> <tr> <td>TURMA 1INF38A</td> <td>MATRICULA:</td> </tr> <tr> <td></td> <td></td> </tr> </table>	TURMA 1INF38A	MATRICULA:		
TURMA 1INF38A	MATRICULA:						

QUESTÃO 1 – [NÍVEL 2] A arquitetura de um sistema tem diversos elementos como: elementos utilitários, elementos de interação, elementos que fazem parte do domínio do problema, elementos de conexão, elementos de persistência, etc. Dessa forma, na arquitetura sempre definimos os seus elementos que precisarão ser utilizados no software e como eles se conectam. No entanto, alguns padrões arquiteturais já foram pensados para resolver problemas corriqueiros. Alguns projetos ou organizações combinam esses padrões arquiteturais, pois atendem melhor às suas necessidades ou deram certo para o seu tipo de aplicação. Por isso é sempre interessante entender as características básicas de cada um dos estilos e escolher ou combinar aqueles que atendem melhor às necessidades de um projeto específico, isso tudo, deve ser feito após uma análise do sistema a ser desenvolvido. Entre as arquiteturas existentes temos: Cliente-servidor, P2P - Peer to Peer, Dados compartilhados, Máquina virtual, Camadas, MVC e muitos outros.

Considere uma aplicação Web em desenvolvimento utilizando Java Servlet e JSP e o design pattern MVC. Nesse contexto, é correto afirmar que **(0,8)**:

- ☒ (A) uma classe "Cliente" que possui os atributos "codigo", "nome" e "renda" e os respectivos métodos getters e setters para esses atributos deve ser representada no componente MODEL do MVC.
- (B) uma classe "ClienteDAO" que possui métodos para acessar o banco de dados e executar instruções SQL deve ser representada no componente CONTROLLER do MVC.
- (C) uma página JSP que gera um formulário de cadastro deve ser representada no componente CONTROLLER do MVC.
- (D) a partir de uma página JSP não é possível acessar diretamente uma classe Data Access Object (DAO) que acessa o banco de dados.
- (E) as classes Servlet geralmente são representadas no componente MODEL do MVC, pois mapeiam as rotas de uma aplicação Web.

QUESTÃO 2 – [NÍVEL 2] No padrão MVC é possível definir grupos de componentes principais: o Model (Modelo), o View (Apresentação) e o Controller (Controle). A utilização do padrão MVC trás como benefício isolar as regras de negócios da lógica de apresentação, a interface com o usuário. Isto possibilita a existência de várias interfaces com o usuário que podem ser modificadas sem que haja a necessidade da alteração das regras de negócios, proporcionando assim muito mais flexibilidade e oportunidades de reuso das classes. Considerando estas informações, pode-se afirmar que deve fazer parte do componente: **(0,7)**

- (A) Controller, uma classe que contém um método com a finalidade de calcular o reajuste de salário dos funcionários.
- (B) View, uma classe que contém um método para persistir o salário reajustado de um funcionário.
- (C) Controller, as animações desenvolvidas em Flash.
- (D) View, as validações necessárias ao sistema, geralmente definidas através de um conjunto de comparações.
- ☒ (E) Model, as classes com métodos conhecidos como setters e getters e que representam tabelas do banco de dados.

QUESTÃO 3 – [NÍVEL 5] O protocolo HTTP é utilizado para permitir a comunicação entre aplicações clientes e aplicações em um servidor Web. O HTTP trabalha como um protocolo "request-response" entre o cliente e o servidor. Assim, um Browser pode ser um Cliente, e uma aplicação hospedada no servidor Web pode ser considerada o Servidor. Para enviar dados através de uma página Web, do Cliente para o Servidor, são utilizados principalmente os métodos do request: GET e POST. Apresente e explique quatro características do método POST, fazendo uma comparação com o método GET. (mínimo de 10 linhas) (1,0).

QUESTÃO 4 – [NÍVEL 3] Em software, os padrões de projeto não são classes nem objeto. Em vez disso, os projetistas usam esses padrões para construir conjuntos de classes e objetos. Para utilizá-los de maneira eficaz, os projetistas precisam se familiarizar com os padrões mais populares e eficazes utilizados pela engenharia de software e conhecer o seu contexto e escopo.

Um exemplo existente e muito utilizado de padrão é o Data Access Object, ou simplesmente DAO. Foi uma solução encontrada para resolver problemas referentes à separação das classes de acesso a banco de dados das classes responsáveis pelas regras de negócio. Já o conceito principal do modelo MVC é utilizar uma solução já definida para separar partes distintas do projeto reduzindo suas dependências ao máximo [GUERRA, E - Design Patterns com Java. Projeto Orientado a Objetos Guiado por Padrões - Casa de Código]. Sobre o padrão Data Access Object (DAO) do catálogo de padrões J2EE, assinale a alternativa que não está correta (0,7):

- (A) um objeto DAO, necessariamente, provê acesso a apenas uma tabela de um SGBD. ✓
- (B) se mais de uma fonte de dados for necessária na aplicação, uma solução possível é criar fábricas de objetos DAO, uma para cada fonte de dados.
- ✗ (C) a interface de um objeto DAO, normalmente, contém operações CRUD (create/read/update/delete). ✓
- (D) é um padrão para construir objetos que serve de ponto de acesso a um mecanismo de armazenamento de dados como, por exemplo, um sistema de gerência de banco de dados (SGBD).
- (E) é um padrão para permitir o encapsulamento dos dados dentro de um Java Bean. ✓

QUESTÃO 5 – [NÍVEL 1] Na fase de projeto damos realmente valor à tecnologia, diferente da fase de análise onde ainda estamos esboçando o problema a ser resolvido. Definimos a plataforma e como os componentes do sistema irão se organizar. Evidentemente que os requisitos ainda são importantes, pois, por exemplo, um sistema Web ou então uma aplicação de tempo real deverá influenciar na arquitetura. Mesmo não possuindo uma definição consensual, afinal diversos livros, artigos ou autores definem o que é arquitetura da sua forma, a arquitetura de software de um sistema computacional pode ser definida como a suas estruturas, que são compostas de elementos de software, de propriedades externamente visíveis desses componentes, e do relacionamento entre eles. Ou seja, a arquitetura define os elementos de software e como eles interagem entre si. O padrão definido pela arquitetura MVC (Model, Control e View) ainda é muito utilizado no desenvolvimento de aplicações Web. Em uma aplicação Web desenvolvida com Java utilizando o padrão MVC, os *servlets* e as classes no padrão DAO são representadas, respectivamente, nos componentes (0,7):

- (A) Controller e View.
- (B) View e Model.

- ✗ (C) Controller e Model. ✗
- (D) Model e Controller.
- (E) View e Controller.

QUESTÃO 6 – [NÍVEL 4] Durante o desenvolvimento de uma aplicação Web em Java é comum a implementação de classes Java Bean, que tendem a representar as entidades do modelo de dados. Essas classes são construídas seguindo o princípio básico da orientação a objetos, o encapsulamento, e possuem uma estrutura muito bem definida e padronizada. Baseado nestas informações, responda as questões a seguir (1,0):

- Cite e explique as partes principais que devem ser implementadas na estrutura destas classes.
- Como um Java Bean implementa o princípio do encapsulamento?

QUESTÃO 7 – [NÍVEL 3] Servlets são classes Java desenvolvidas de acordo com uma estrutura bem definida que, quando instaladas e configuradas em um servidor que implemente um "Servlet Container", podem tratar requisições recebidas de clientes Web, como por exemplo os Browsers [DEITEL, P.; DEITEL, H. - Como Programar Java - Pearson - 10 ed.]. Considere as seguintes afirmações sobre as classes servlets (0,8):

- I. Herdam a classe `HttpServletRequest` e implementam as classes `HttpServletRequest` e `HttpServletResponse`.
- II. Em aplicações que utilizam o padrão Model-View-Controller, são representadas no grupo Controller.
- III. Normalmente possuem métodos para receber requisições que foram enviadas utilizando os métodos HTTP Get e Post.
- IV. Utilizam classes dos pacotes `javax.servlet`, `javax.swing` e `java.io`.

Está correta o que se afirma somente em:

- (A) As asserções I e III são proposições corretas e as demais são falsas.
- (B) As asserções II e IV são proposições corretas e a I e a III são falsas.
- (C) Apenas a asserção II é uma proposição correta.
- (D) Apenas a asserção III é uma proposição correta.
- ☒ (E) Apenas as asserções II e III são proposições corretas.

QUESTÃO 8 – [NÍVEL 1] No desenvolvimento de aplicações Web, é comum o uso de Servlets. Na plataforma Java, a primeira e principal tecnologia capaz de gerar páginas dinâmicas são as Servlets, que surgiram no ano de 1997. Hoje, a versão mais encontrada no mercado é baseada nas versões 2.x, mais especificamente a 2.4 (parte do J2EE 1.4) e a 2.5 (parte do Java EE 5). Desta forma, é possível afirmar que um Servlet é um programa que executa (0,8):

- ☒ (A) apenas uma vez, gerando uma página JSP que é reutilizada muitas vezes pelo servidor web.
- (B) dentro de uma página web no navegador do usuário.
- (C) dividido em duas partes, uma executando no servidor, outra, no navegador, por meio de chamadas RPC.
- (D) em um banco de dados e retorna os dados solicitados por uma página web.
- ☒ (E) em um servidor web, a cada chamada construindo uma página web que é enviada ao usuário.

QUESTÃO 9 – [NÍVEL 6] Durante as aulas da disciplina "Aplicações na Internet" desenvolvemos uma aplicação Web completa, utilizando vários conceitos e soluções tecnológicas, dentre os quais podemos destacar (3,5):

- No Frontend: HTML (4.0 e 5.0), CSS, Java Script, JQuery, Bibliotecas Validation e Masked input e o Bootstrap.
- No Backend: a linguagem Java, o modelo MVC, Java Bean, o design pattern DAO, os métodos get e post, servlet, projeto e implementação do banco de dados da aplicação, JSTL e JDBC.

Considerando os conceitos e tecnologias aprendidas e as partes de código apresentadas a seguir, desenvolva os itens abaixo solicitados:

- O form para a inclusão de um empregado (1,0)
- O método de inclusão no Servlet EmpregadoControle (1,5)
- O método de consulta a um empregado na classe EmpregadoDao (1,5)

CLASSE EMPREGADO

```
package model.entity;

public class Empregado {
    private Long cpf;
    private String nome;
    private String sexo;
    private String dtAdmissao;
    private Double salario;

    public Empregado() {
        super();
    }

    public Empregado(Long cpf, String nome, String sexo, String dtAdmissao, Double
salario) {
        super();
        this.cpf = cpf;
        this.nome = nome;
        this.sexo = sexo;
        this.dtAdmissao = dtAdmissao;
        this.salario = salario;
    }

    public Long getCpf() {
        return cpf;
    }

    public void setCpf(Long cpf) {
        this.cpf = cpf;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getSexo() {
        return sexo;
    }

    public void setSexo(String sexo) {
        this.sexo = sexo;
    }

    public String getDtAdmissao() {
        return dtAdmissao;
    }

    public void setDtAdmissao(String dtAdmissao) {
        this.dtAdmissao = dtAdmissao;
    }

    public Double getSalario() {
        return salario;
    }

    public void setSalario(Double salario) {
        this.salario = salario;
    }
}
```

SERVLET EMPREGADOCONTROLE

```

package control;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import model.entity.Empregado;
import model.persistence.EmpregadoDao;

@WebServlet({"/ControleEmpregado", "cad_empregado.html", "cons_empregado.html" })
public class ControleEmpregado extends HttpServlet {

    private static final long serialVersionUID = 1L;

    public ControleEmpregado() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        execute(request, response);
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        execute(request, response);
    }

    protected void execute(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        try{
            String url = request.getServletPath();

            if(url.equalsIgnoreCase("cad_empregado.html")){
                cadastrar(request, response);
            }

            if(url.equalsIgnoreCase("cons_empregado.html")){
                consultar(request, response);
            }

            }else{
                response.sendRedirect("/");
            }

        }catch(Exception e){
            e.printStackTrace();
        }
    }

    protected void consultar(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        try{
            Long cpf = new Long(request.getParameter("cpf"));
            EmpregadoDao ed = new EmpregadoDao();
            Empregado empregado = ed.buscarPorCpf(cpf);

            if(empregado == null){

```

```

        request.setAttribute("msg", "<div class='alert alert-
warning'>Empregado nao existente</div>");

        request.getRequestDispatcher("ConsultarEmpregado.jsp").forward(request,
response);
    }else{
        request.setAttribute("c", empregado);

        request.getRequestDispatcher("CadastraroEmpregado.jsp").forward(request,
response);
    }

    }catch(Exception e){
        e.printStackTrace();
    }
}
}

```

CLASSE DAO

```

package model.persistence;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Dao {

    Connection con;
    PreparedStatement stmt;
    ResultSet rs;

    private final String URL = "jdbc:mysql://localhost:3306/lojaonline";
    private final String USER = "root";
    private final String PASS = "123456";

    protected void open() throws Exception{
        Class.forName("com.mysql.jdbc.Driver");
        con = DriverManager.getConnection(URL, USER, PASS);
    }

    protected void close() throws Exception{
        con.close();
    }
}

```

CLASSE EMPREGADODAO

```

package model.persistence;
import java.sql.SQLException;
import model.entity.Empregado;

public class EmpregadoDao extends Dao{

    public boolean cadastrar(Empregado e) throws Exception{
        boolean success = false;
        try{
            open();
            stmt = con.prepareStatement("insert into empregado values(?, ?, ?, ?, ?)");
            stmt.setInt(1, e.getCpf());
            stmt.setString(2, e.getNome());
            stmt.setString(3, e.getSexo());

```

UVA

```
        stmt.setString(4,e.getDtAdmissao());
        stmt.setDouble(5,e.getSalario);
        stmt.execute();
        success = true;

    }catch(SQLException e1){
        e1.printStackTrace();
    }finally{
        close();
    }
    return success;
}

}
```